

MANUAL TECNICO DE SABOR EXPRESS

Juan Manuel Pérez

Yilmer Nicolas Nieto

Héctor Hernán Díaz Guzmán

Junio 2026

1. Contexto

Durante la etapa de diseño técnico y desarrollo del sistema de gestión para el restaurante "La Barra del Sabor", nos enfocamos en estructurar el modelo lógico y escribir los scripts SQL en MySQL Workbench. Nos dimos cuenta de que necesitábamos definir una regla para nombrar las tablas, columnas, llaves primarias y foráneas. Si cada uno escribía a su manera, el código de la base de datos estaría desordenado, complicando las consultas, y el desarrollo del mismo.

Teníamos opciones de CamelCase o snake_case. El reto principal era elegir una opción que mantuviera la legibilidad.

2. Decisión base de datos en MySQL

Tomamos la decisión de utilizar la convención snake_case para absolutamente todos los elementos del modelo lógico y la estructura física de la base de datos en MySQL.

3. Justificación

Elegimos snake_case basándonos en razones prácticas que nos facilitan el desarrollo del proyecto:

- **Compatibilidad real con motores SQL:** Motores como MySQL o Apache NetBeans pueden llegar a ser propensos a problemas de distinción entre mayúsculas y minúsculas. Usar solo minúsculas con guiones bajos nos da tranquilidad y evita fallos al migrar o montar el entorno local.
- **Legibilidad en nombres compuestos largos:** En un sistema de restaurante manejamos términos complejos. Leer fecha_vencimiento_clave o id_estado_pedido de un solo vistazo es más natural para nosotros que intentar descifrar bloques de texto planos o cadenas pegadas con mayúsculas.
- **Estandarización del equipo:** Al ser tres integrantes en el equipo de desarrollo, definir que todo va separado por guiones bajos automatizó la escritura de las restricciones de llaves foráneas (como fk_usuarios_tipos_documento1_idx o metodos_pago_id_metodo_pago), asegurando que el código final de MySQL Workbench se vea uniforme y limpio.

Ejemplo Base de datos

```
CREATE DATABASE IF NOT EXISTS sabor_express;  
USE sabor_express;
```

```
CREATE TABLE roles (  
    id_rol INT(11) NOT NULL,  
    tipo_de_rol VARCHAR(45) NOT NULL,  
    estado TINYINT(1) NOT NULL DEFAULT 1,  
    PRIMARY KEY (id_rol)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
CREATE TABLE permisos (  
    id_permisos INT(11) NOT NULL AUTO_INCREMENT,  
    descripcion VARCHAR(45) NOT NULL,  
    estado TINYINT(1) NOT NULL DEFAULT 1,  
    PRIMARY KEY (id_permisos)  
) ENGINE=InnoDB AUTO_INCREMENT=5 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
CREATE TABLE tipos_documento (  
    id_tipo_documento INT(11) NOT NULL AUTO_INCREMENT,  
    descripcion_tipo_documento VARCHAR(45) NOT NULL,  
    estado TINYINT(1) NOT NULL DEFAULT 1,  
    PRIMARY KEY (id_tipo_documento)  
) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

4. Consecuencias

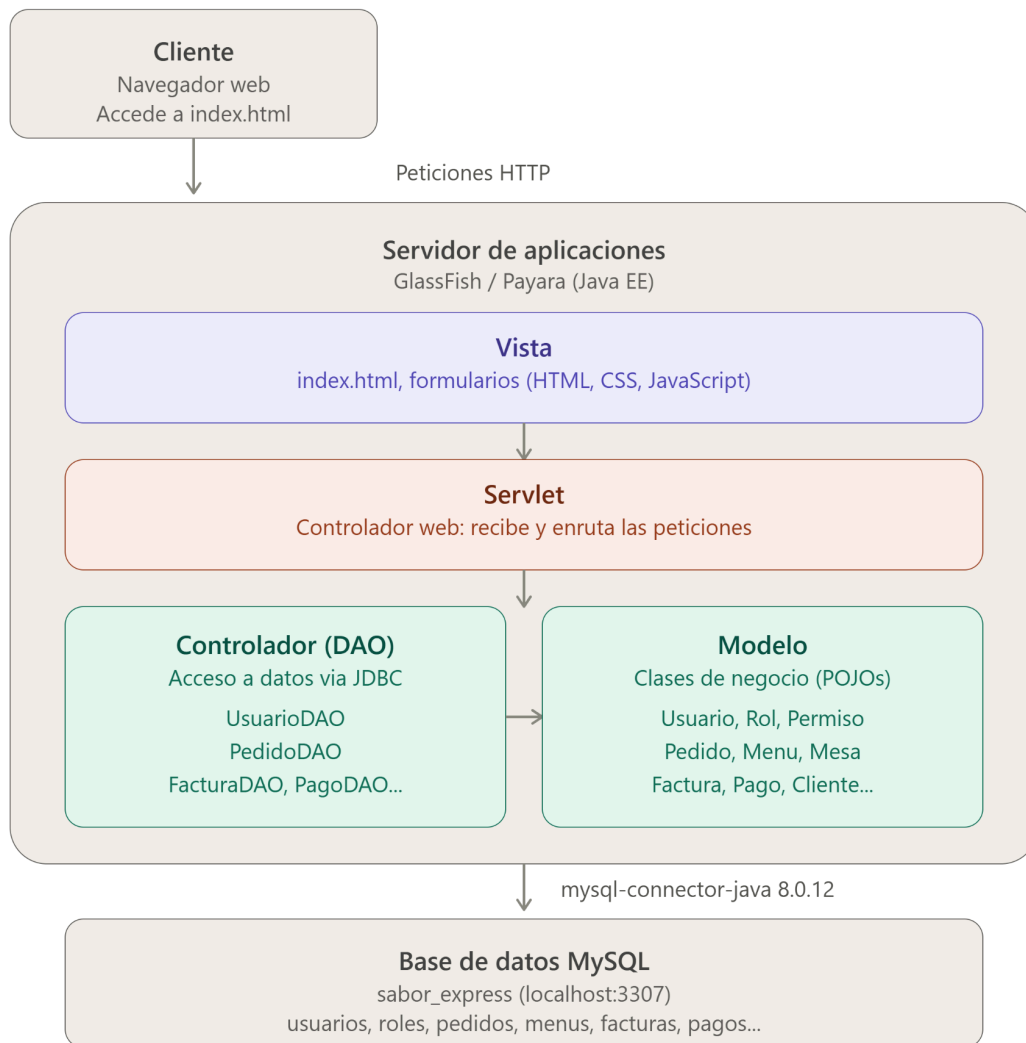
Consecuencias Positivas

- Los scripts de creación (CREATE TABLE, ALTER TABLE) quedaron estandarizados y limpios.
- Es sumamente sencillo mapear las entidades hacia la base de datos, ya que no hay confusión con nombres de columnas ambiguos.
- Mayor facilidad a la hora de escribir y depurar las consultas, así como relacionar los pedidos con los clientes y las mesas del local.

Consecuencias Negativas / Esfuerzo Requerido

- Nos exige ser rigurosos y revisar bien los nombres al momento de pasar los requerimientos de la lógica de negocio al script SQL, asegurándonos de que todo esté escrito con la convención del snake_case.

Diagrama de arquitectura



¿Porqué se escogió la arquitectura de MVC?

La adopción de la arquitectura Modelo-Vista-Controlador (MVC) en Sabor Express se justificó principalmente por la necesidad de separar las responsabilidades del sistema para facilitar el desarrollo en equipo y el mantenimiento a largo plazo. Al dividir el software en tres capas independientes, logramos que la lógica de negocio (Modelo), la interfaz de usuario para el restaurante (Vista) y la gestión de peticiones como la creación de comandas o facturas (Controlador) no dependieran directamente entre sí. Esto nos permitirá trabajar de forma simultánea en el backend y el frontend sin interferir en el código de los demás integrantes.

Además, esta estructura modular garantiza la escalabilidad del proyecto ante futuros requerimientos del negocio, como la integración de nuevos métodos de pago o un módulo avanzado de inventarios. Al estar la lógica de datos totalmente aislada de la interfaz gráfica, cualquier cambio en las vistas de la aplicación o en las consultas de MySQL se puede implementar de manera rápida y segura, disminuyendo drásticamente el riesgo de introducir errores en otras áreas del sistema y facilitando las futuras pruebas técnicas.

Especificación de Tecnologías Backend, Frontend y Entorno de Desarrollo

- **Backend**

Servidor de Aplicaciones / Runtime: Eclipse GlassFish v7.0.0
(compatible con Jakarta EE 10).

Lenguaje de Programación: Java

Base de Datos: MySQL

Conector BD: JDBC Driver for MySQL

- **Frontend**

Tecnologías Base: HTML5 / CSS3.

Framework de Estilos: Bootstrap v5

- **Herramientas de Desarrollo (Entorno)**

IDE (Entorno de Desarrollo Integrado): Apache NetBeans IDE 20.

Link Repositorio: https://github.com/Juan-P01/PROYECTO_SABOR-EXPRESS